

Data Structures & Algorithms (CSC-214)

Lecture 14: Stack Applications

Matching Tags in an HTML Document

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even
as a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

(a)

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

(b)

Figure 5.3: HTML tags: (a) an HTML document; (b) its rendering.

Input

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even
as a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

Output

`<body>`, `<center>`, `<h1>`, `</h1>`, `</center>`, ..., `</body>`

String find()

Function Template:

- `size_t find (const string& str, size_t pos = 0);`
- `size_t find (const char* s, size_t pos = 0);`

Function parameters:

- `str` : The sub-string to be searched.
- `s` : The sub-string to be searched, given as C style string.
- `pos` : The initial position from where the string search is to begin.

Function Return:

- The function returns the index of the first occurrence of sub-string, if the sub-string is not found it returns `string::npos` (`string::pos` is static member with value as the highest possible for the `size_t` data structure).

geeksforgeeks



```
int main()
{
    string str = "geeksforgeeks a computer science";
    string str1 = "geeks";

    // Find first occurrence of "geeks"
    size_t found = str.find(str1);
    if (found != string::npos)
        cout << "First occurrence is " << found << endl;

    // Find next occurrence of "geeks". Note here we pass
    // "geeks" as C style string.
    char arr[] = "geeks";
    found = str.find(arr, found+1);
    if (found != string::npos)
        cout << "Next occurrence is " << found << endl;

    return 0;
}
```

Output:

```
First occurrence is 0
Next occurrence is 8
```

String substr

Syntax:

```
string substr (size_t pos, size_t len) const;
```

Parameters:

pos: Position of the first character to be copied.

len: Length of the sub-string.

size_t: It is an unsigned integral type.

Return value: It returns a string object.



```
// CPP program to illustrate substr()
```



```
#include <string.h>
```



```
#include <iostream>
```



```
using namespace std;
```

```
int main()
```

```
{
```

```
    // Take any string
```

```
    string s1 = "Geeks";
```

```
    // Copy three characters of s1 (starting
```

```
    // from position 1)
```

```
    string r = s1.substr(1, 3);
```

```
    // prints the result
```

```
    cout << "String is: " << r;
```

```
    return 0;
```

```
}
```

Output:

```
String is: eek
```

Input

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even
<body>, <center>, <h1>, </h1>, </center>, ..., </body>
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

Output

```
<body>, <center>, <h1>, </h1>, </center>, ..., </body>
```

```

vector<string> getHtmlTags() {           // store tags in a vector
    vector<string> tags;                 // vector of html tags
    while (cin) {                        // read until end of file
        string line;
        getline(cin, line);             // input a full line of text
        int pos = 0;                     // current scan position
        int ts = line.find("<", pos);     // possible tag start
        while (ts != string::npos) {     // repeat until end of string
            int te = line.find(">", ts+1); // scan for tag end
            tags.push_back(line.substr(ts, te-ts+1)); // append tag to the vector
            pos = te + 1;                  // advance our position
            ts = line.find("<", pos);
        }
    }
    return tags;                         // return vector of tags
}

```

Code Fragment 5.12: Get a vector of HTML tags from the input, and store them in a vector of strings.

```

// check for matching tags
bool isHtmlMatched(const vector<string>& tags) {
    LinkedStack S; // stack for opening tags
    typedef vector<string>::const_iterator lter; // iterator type
    // iterate through vector
    for (lter p = tags.begin(); p != tags.end(); ++p) {
        if (p->at(1) != '/') // opening tag?
            S.push(*p); // push it on the stack
        else { // else must be closing tag
            if (S.empty()) return false; // nothing to match - failure
            string open = S.top().substr(1); // opening tag excluding '<'
            string close = p->substr(2); // closing tag excluding '</'
            if (open.compare(close) != 0) return false; // fail to match
            else S.pop(); // pop matched element
        }
    }
    if (S.empty()) return true; // everything matched - good
    else return false; // some unmatched - bad
}

```

Code Fragment 5.13: Check whether HTML tags stored in the vector tags are matched.

Post Fix Notation

- Postfix notation, also known as Reverse Polish Notation (RPN), is a way of writing mathematical expressions without the use of parentheses to indicate the order of operations
- In postfix notation, operators are written after their operands
- This eliminates the need for parentheses and makes the evaluation of expressions straightforward using a stack-based approach

$$(3 + 4) * (5 / 2)$$

$$3\ 4\ +\ 5\ 2\ /\ *$$

Read in the expression character-by-character. As each character is read in:

- If the character corresponds to a single digit number (characters '0' to '9'), then push the corresponding floating-point number onto the stack.
- If the character corresponds to one of the arithmetic operators (characters '+', '-', '*', and '/'), then
- Pop a number off of the stack. Call it *operand1*.
- Pop a number off of the stack. Call it *operand2*.
- Combine these operands using the arithmetic operator, as follows:

$Result = operand2 \text{ operator } operand1$

- Push *result* onto the stack.

When the end of the expression is reached, pop the remaining number off the stack. This number is the value of the expression.

3 4 + 5 2 / *

yields the following computation

'3' : Push 3.0

'4' : Push 4.0

'+' : Pop, *operand1* = 4.0

Pop, *operand2* = 3.0

Combine, *result* = 3.0 + 4.0 = 7.0

Push 7.0

'5' : Push 5.0

'2' : Push 2.0

'/' : Pop, *operand1* = 2.0

Pop, *operand2* = 5.0

Combine, *result* = 5.0 / 2.0 = 2.5

Push 2.5

'*' : Pop, *operand1* = 2.5

Pop, *operand2* = 7.0

Combine, *result* = 7.0 * 2.5 = 17.5

Push 17.5

'\n' : Pop, Value of expression = 17.5

More examples

Expression (postfix)	For your reference only	Result
9 3 /	$9 / 3$	3
6 2 / 5 +	$(6 / 2) + 5$	8
4 5 7 2 + - *	$4 * (5 - (7 + 2))$	-16
4 2 3 5 1 - + * +	$4 + 2 * (3 + 5 - 1)$	18
4 2 + 3 5 1 - * +	$4 + 2 + 3 * (5 - 1)$	18
3 4 + 2 * 7 /	$(3 + 4) * 2 / 7$	2
1 3 + 2 3 - * 1 /	$(1 + 3) * (2 - 3) / 1$	-4

*Lecture content adapted from Michael T. Goodrich textbook,
chapter 5 & also the lab text book, stacks chapter.*